

Containerization and Orchestration of Software for Autonomous Mobile Robots: a Case Study of Mixed-Criticality Tasks across Edge-Cloud Computing Platforms

Francesco Lumpp^α, Franco Fummi^α, Hiren D. Patel^β and Nicola Bombieri^α

Abstract—Containerization promises to strengthen platform-independent development, better resource utilization, and secure deployment of software. As these benefits come with negligible overhead in CPU and memory utilization, containerization is increasingly being adopted in mobile robotic applications. An open challenge is supporting software tasks that have mixed-criticality requirements. Even more challenging is the combination of real-time containers with orchestration, which is an emerging paradigm to automate the deployment, networking, scaling, and availability of containerized workloads and services. This paper addresses this challenge by presenting a framework that extends the de-facto reference standard for container orchestration, Kubernetes, to schedule tasks with mixed-criticality requirements. Quantitative experimental results on the software implementing the mission of a Robotnik RB-Kairos mobile robot demonstrate the effectiveness of the proposed approach. The source code is publicly available on GitHub.

I. INTRODUCTION

Container-based virtualization is an emerging trend to strengthen platform-independent development and provisioning of secure and isolated software applications. *Containerization* encapsulates application functions and their dependencies into virtual environments (i.e., containers) to allow runtime isolation on a shared operating system kernel. Besides flexibly consolidating automation functions on different hardware types and scales, containers are also used to extend the life-time of legacy code [1].

Distinct from standard hardware virtualization, which requires a hypervisor and introduces considerable performance overhead [2], such an OS-level virtualization provides near-native performance, rapid deployment times, and a low overhead while still retaining resource isolation and control [3]. For all these reasons, containerization platforms (e.g., *Docker*, *AWS Fargate*, *LXC*, *Singularity*) are increasingly being adopted also for software deployment onto distributed and resource-constrained architectures in industrial, cyber-physical, as well as automotive applications [4].

On the other hand, an increasing number of SW standards, including automotive [5], avionics [6], and industrial automation [7], recognize and support the concepts of mixed-criticality. A mixed-criticality system (MCS) integrates software components (i.e., ROS nodes) with different levels of criticality into a common hardware platform [8]. One of the main research challenges for MCSs is to guarantee the correct execution of high-critical tasks with a user-transparent use of the underlying resources, which are shared with lower- or non-critical tasks [9]¹. In this context, the software components and layers introduced by containerization can conflict with real-time requirements [10]. Even though some research effort has been invested to bring real-time properties in container-based virtualization [11], [12], [13], supporting tasks that have mixed-criticality requirements is still an open challenge.

^α Dept. of Computer Science, Univ. of Verona, Italy

^β Dept. of Electrical and Computer Eng., Univ. of Waterloo, Canada

This work has been supported by the project Dipartimenti di Eccellenza 2018-2022 funded by the Italian Ministry of Education, Univ. and Research.

¹For the sake of clarity, we refer to ROS nodes as software tasks (or tasks), while computing nodes of the Edge-Cloud cluster as nodes.

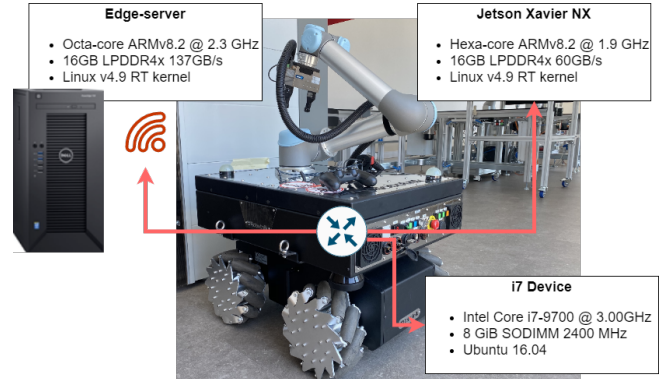


Fig. 1: RB-Kairos compute architecture.

Even more challenging is the combination of *real-time containers* with *orchestration*. In the context of Cloud-native applications, orchestration has been proven to be a key paradigm to automate the deployment, networking, scaling, and availability of containerised workloads and services [14]. Kubernetes (K8S) has become the de-facto reference standard for software deployment in Cloud Computing through container orchestration. On the other hand, due to the advances on the Edge-Fog-Cloud continuum, the orchestration concept is increasingly being adopted also in resource constrained and distributed computing architectures (e.g., *K3S*, *Kubedge*) [15], [16]. In the automotive domain, from Tesla to Ford to Mercedes-Benz, major car manufacturers are leveraging Kubernetes to build intelligent vehicles whose technology will continue evolving with Kubernetes in the future [17].

This paper addresses the challenge of containerization and orchestration of software for autonomous mobile robots, which is characterized by mixed criticality tasks deployed across Edge-Cloud computing architecture (see Fig. 1). It proposes a platform that extends the standard Kubernetes to support orchestration of tasks with real-time constraints. The platform introduces the real-time specifications of time-sensitive tasks (i.e., deadline, period, worst-case execution time), in addition to the standard task specifications, to be considered during the task scheduling onto the cluster of computing nodes. It implements schedulability analysis for real-time tasks and supports different criticality levels to implement isolation and limit task interference. The paper presents the results obtained by applying the proposed platform to orchestrate different containerized mixed-criticality software benchmarks and a real-case of study implementing the mission of a Robotnik RB-Kairos mobile robot for navigation and transportation of goods.

The paper is organized as follows. Section II presents some background and the related work. Section III presents the proposed platform. Section IV presents the experimental results, while Section V presents the conclusions.

II. RELATED WORK

Containerization in Edge-Fog-Cloud architectures has been investigated in several works (e.g., [18], [19], [20]). The experimental results show that container-based virtualization combined to the

Edge-Cloud computing continuum provides better scalability, resource utilization, and performance. Containers bring little to no performance overhead, even though care is needed when multiple containers access shared resources [21]. Different generic architectures for cyber-physical systems (CPSs), based on containers (e.g., Docker), using ROS as a middleware have been recently explored [22], and the modular architecture revealed the potential for better information flow among different network levels as well as increased modularity in the use of software components.

In [4], the authors address the problem of applying containerization on a Fog-based system for robotic applications. They underline the absence of frameworks to support the real-time requirements of robotic applications. Due to the increasing adoption of container-based virtualization, containers with real-time constraints (i.e., RT-containers) has recently been the focus of several research work. The review in [23] explores existing solutions that guarantee real-time constraints when working with containerized applications.

In [10] the authors show how legacy applications with real-time constraints can be successfully emulated inside containers. In [7] the authors analyze the performance overhead of containers in real-time applications, showing that containers allow for the stringent requirements of industrial automation applications. In [12], the authors propose an extension of the Linux kernel to improve the real-time scheduling of *cgroups* (i.e., the main technology used to enable containers). In [13], the authors found that containerized real-time applications are handled better by the Linux kernel when using the *SCHED_DEADLINE* policy, rather than limit resources with the *cgroup* policies. In [24], the authors test various real-time technologies (real-time OS ResinOs, Ubuntu Core, Co-kernel Xenomai 3, and Ubuntu with Preempt-rt) to support containerization. In a follow up work ([25]), they propose a framework to schedule real-time containers for High Performance Computing (HPC). They note that further work is required to calculate OS and I/O impact on deadlines.

In [11], the authors present a framework that allows applications in mixed criticality systems to not interfere with each other when an application exceeds its Worst Case Execution Time (WCET). In [26], the authors analyze the advantages of leveraging both the SGX isolation and the unikernel features to meet the requirements of safety-critical real-time systems and ease the certification process. In [27], the authors present REACT, a real-time orchestration plugin that extends Kubernetes and takes advantage of a custom patch for the Linux kernel to deploy best effort and real-time tasks.

Differently from all the works in literature, this paper presents a platform for container orchestration onto Edge-Cloud architectures for mixed-criticality systems, which is based on off-the-shelf technology (i.e., Linux OS + Preempt-RT patch, Kubernetes-K3S). To satisfy functional and non-functional constraints of advanced and multi-domain software, which is typical of autonomous mobile robots, the platform supports different levels of criticality (e.g., A, B, and C for RT containers). The platform has been implemented as a lightweight, edge-focused, K3S-Kubernetes plugin and can be downloaded from github (<https://github.com/FrancescoL96/k3s>).

III. METHODOLOGY

Fig. 2 presents an overview of the proposed orchestration platform with support for real-time containers (RT-containers). The orchestration begins with the queue of containers (i.e., standard and RT), and the list of nodes representing the compute platforms (i.e., the *cluster* of computing devices). We assume that each real-time task (RT task) is mapped to one RT-container. Our evaluation

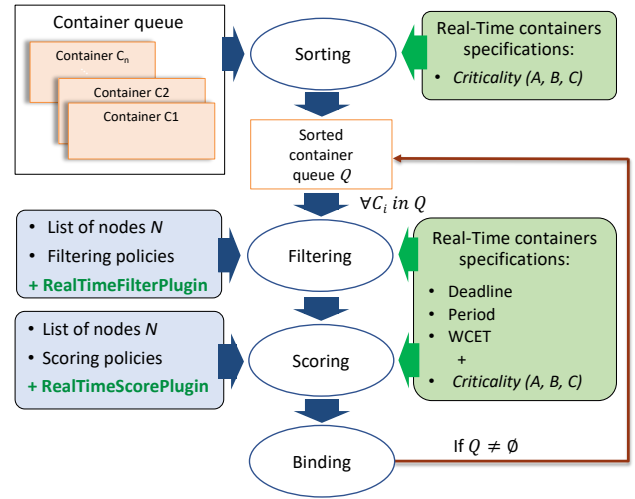


Fig. 2: The proposed orchestration platform with support for real-time containers.

shows that this one-to-one configuration when compared to other solutions is the most flexible, it incurs negligible overhead, and it has little performance penalties for RT tasks (Section IV-A).

A set of specifications is provided for each container (e.g., memory, CPU, storage requirements). Specifications for RT-containers are extended to include temporal information for the real-time schedulability check (see Section III-A). The platform implements four main steps. First, *sorting* of containers using a hierarchical scheduling policy to generate a priority queue ordered by criticality. Next, for each container in the queue, the *filtering* phase selects the nodes of the cluster that satisfy the container specifications (Section III-B). The *scoring* phase creates a node ranking by considering user-defined policies, and schedules the container on the highest ranking node (Section III-C). Finally, the *binding* phase maps the container to the node by allocating resources for the container.

A. Extending container specifications

To schedule RT-containers, four additional fields are included in the container specifications: The criticality, the deadline, the runtime (WCET), and the period of the corresponding containerized RT task. These fields are used to calculate the utilization for all the containers on a single node, and assess the performance impact of a new RT container deployment.

Listing 1 shows an example of the extended specifications for the deployment of the Kubernetes *nginx* use case Lines 15 to 19 give the real time information with an additional block (i.e., *realTime*) containing the four additional values. *Criticality* is used for the new queue and score phases. It is a string that can be set to one of the Automotive Safety Integrity Levels (ASIL) A, B or C, with C being the highest the platform supports as of now. Note that it would be possible to support ASIL-D tasks, but it would require a compute platform that honours ASIL-D requirements as well.

The other three are for the filtering phase and represent the deadline, period and WCET. To use these values, the platform relies on the *RT-scheduler* (line 9), which is a custom configuration of the Kubernetes scheduler with the plugins for sorting, filtering, and scoring enabled. To deploy RT-containers, the platform needs to know the capabilities of the deployment node. This is accomplished via the *taint* and *tolerations* features to identify which nodes have a real-time capable kernel installed (line 20). If a node does not have a RT operating system or RT capabilities, then it is automatically excluded from the pool of schedulable nodes for RT

```

1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: nginx-deployment
5   labels:
6     app: nginx
7   [...]
8   spec:
9     schedulerName: RT-scheduler
10    containers:
11      - name: nginx
12        image: nginx:1.14.2
13        ports:
14          - containerPort: 80
15        realTime:
16          criticality: A
17          rtDeadline: 100
18          rtPeriod: 100
19          rtWcet: 95
20    tolerations:
21      - key: "RealTime"
22        operator: "Equal"
23        value: "RT"
24        effect: "NoSchedule"

```

Listing 1: Kubernetes deployment configured with RT parameters

tasks. The task characteristics are used in the filtering phase, while the criticality is used in sorting and scoring (see Fig. 2).

B. Filtering of nodes

When scheduling any RT task with the *schedule deadline* policy, Linux uses an admission test to check if there is room to guarantee the task *runtime* (WCET) within the deadline, every period. This means that the task (t_{new}) has to pass the following *task admission test* to be placed in the scheduling pool of RT tasks:

$$\sum_i^{RT \cup \{t_{new}\}} \frac{WCET_i}{P_i} \leq M * \frac{sched_rt_runtime_us}{sched_rt_period_us} \quad (1)$$

where RT is the set of running RT tasks, $WCET_i$ and P_i are the worst-case execution time and period of the tasks, M is the number of CPU cores, and $sched_rt_runtime_us/sched_rt_period_us$ represents the maximum allowed utilization of the CPU for RT tasks (user-defined Linux kernel variable equal to 95% by default).

The platform applies eqn. 1 extended to the containerized version of tasks to implement the *container admission test*. Algorithm 1 implements the filtering phase, which relies on such a container admission test. The algorithm takes as input the list of nodes N of the cluster, and the container of task t_{new} that has to be scheduled, x . For each node of the cluster (line 3), the algorithm considers all the containers currently running (i.e., already deployed) in the node (i.e., $c \in n$) and sums up the utilization of each container (line 4). The result ($currentUT_n$) represents the left hand side of eqn. 1 extended to containers. The algorithm calculates the projected total utilization of the node ($newUT_n$) by considering the additional resources of container x under deployment (line 5). If the resulting projected utilization is greater than the threshold $n.thresholdUT$ (which represents the right hand side of eqn. 1 extended for containers), the algorithm filters the current node from the pool of schedulable nodes. The algorithm implements an XOR operator (line 8) to check such a node schedulability by verifying whether both the node and task criticality are C . If only one of the two is C , the node n is marked as not schedulable.

C. Scoring of nodes

To implement the scoring phase, the platform relies on the following equation to obtain a normalized ranking ($nRank$) for

Algorithm 1: Real-time filtering extension for the Kubernetes scheduler.

input : A list of nodes N , an RT-container x
output: A list of schedulable nodes M

```

1 Function RealTimeFilterPlugin ( $N, x$ ):
2    $M \leftarrow N$ 
3   for  $\forall n \in N$  do
4      $currentUT_n \leftarrow \sum_{c \in n} \frac{WCET_c}{P_c}$ 
5      $newUT_n \leftarrow currentUT_n + \frac{WCET_x}{P_x}$ 
6     if  $newUT_n > n.thresholdUT$  then
7        $M \leftarrow M - \{n\}$ 
8     else if ( $x.criticality = "C" \oplus n.Labels[RTCriticality]$ 
9        $= "C"$ ) then
10       $M \leftarrow M - \{n\}$ 
11    end
12 return  $M$ 

```

each node:

$$\forall n \in N : nRank = \begin{cases} 1 - \frac{n.thresholdUT - newUT_n}{n.thresholdUT} & \text{if crit.} = A \\ \frac{n.thresholdUT - newUT_n}{n.thresholdUT} & \text{if crit.} = C \end{cases} \quad (2)$$

where $n.thresholdUT$ represents the threshold RT utilization ($M * sched_rt_runtime_us / sched_rt_period_us$), and $newUT_n$ represents the projected RT utilization after the deployment of x in n .

With a normalized utilization value $\in [0, 1]$ for each node, independent of the total runtime and CPU cores available on the node, the platform applies a custom policy for scoring based on the value of the *criticality* field in the RT specification extension (see Algorithm 2). If a task has a criticality level of A, the algorithm assigns the task to the node with the highest RT load (i.e., the node with the highest normalized utilization), line 3. In contrast, for level C, the algorithm gives the highest rank to the node with the lowest normalized utilization (line 11). For the criticality level B (line 5), the algorithm maps the utilization to a function that gives the highest rank to the nodes with a utilization level closest to $\frac{1}{K}$, as follows:

$$\forall n \in N : nRank = \begin{cases} h\left(\frac{n.thresholdUT - newUT_n}{n.thresholdUT}\right) & \frac{n.thresholdUT - newUT_n}{n.thresholdUT} \leq \frac{1}{K} \\ g\left(\frac{n.thresholdUT - newUT_n}{n.thresholdUT}\right) & \text{otherwise} \end{cases} \quad (3)$$

Scoring with this function allows us to modify the deployment behaviour to best fit the needs and requirements of the tasks. For example, we can make a criticality B task resemble the behaviour of a criticality A task with $K \rightarrow 1^-$ and a linearly increasing $h(\cdot)$, or closer to C with $K \rightarrow 0^+$ and a linearly decreasing $g(\cdot)$.

In our experimental analysis, we applied equation 3 with $K = 2$, a linear function $h(\cdot) = a(\cdot) + b$, and a quadratic function for $g(\cdot) = a'(\cdot)^2 + b'(\cdot) + c'$. This allows us to have linearly increasing rank for nodes that would have $newUT_n$ lower than $\frac{1}{2}$ but greater than 0, and then a sharp decrease once that the threshold utilization value is reached.

IV. RESULTS

We evaluated the impact of the containerization overhead on RT tasks in terms of WCET overruns and missed deadlines (Section IV-A). We compared the efficiency of the proposed Kubernetes extension to support RT-containers in terms of task rejections and missed deadlines. We present the results obtained with standard RT benchmarks and with the real case of study (Sections IV-B, IV-C).

Algorithm 2: Real-time scoring extension for the Kubernetes scheduler.

input : node n , RT container x , projected UT newUT_n
output: The score s for RT container x on node n

```

1 Function RealTimeScorePlugin ( $n, x, \text{newUT}_n$ ):
2    $\text{nRank} \leftarrow \frac{n.\text{thresholdUT} - \text{newUT}_n}{n.\text{thresholdUT}}$ 
3   if  $x.\text{criticality} = \text{"A"}$  then
4      $s \leftarrow 1 - \text{nRank}$ 
5   else if  $x.\text{criticality} = \text{"B"}$  then
6     if  $\text{nRank} \leq \frac{1}{K}$  then
7        $s \leftarrow h(\text{nRank})$ 
8     else
9        $s \leftarrow g(\text{nRank})$ 
10    end
11  else if  $x.\text{criticality} = \text{"C"}$  then
12     $s \leftarrow \text{nRank}$ 
13  end
14 return  $s$ 

```

A. Containerization impact on RT-tasks

For the experimental analysis, we used an embedded off-the-shelf system composed by an Nvidia Jetson Xavier (i.e., 8 core ARMv8.2 CPU, 16GB DRAM) with JetPack 4.5, Linux kernel version 4.9, and the *preempt_rt* patch.

Workload configuration. For the sake of space, we only report the results obtained with the *cyclicdeadline* benchmark of *rt - tests* (the results with the other benchmarks show equivalent behaviour) configured to run with either 1, 2, 4 or 8 identical tasks, maximum utilization of 95% per task, WCET of 3.8ms, period of 4.0ms, and deadline equal to the period (i.e., we set the system very close to the full workload capacity). We evaluated three scenarios: (1) All RT tasks running natively on the edge platform; (2) All RT tasks running in one Docker container; (3) All RT tasks running, where each RT task is mapped into a single Docker container. We run the scenarios with more tasks that overload the system with memory accesses to assess the behaviour of the RT tasks under stress.

Key results. Table I shows the results in terms of average runtime among the n tasks, observed WCET among all the n tasks, total WCET overruns, and total of missed deadlines for all n tasks.

We found that containerization does introduce overhead in terms of observed WCET, and that such an overhead is evident only with 4 or more tasks on the 8-core CPU. Containerization does not impact the average runtime of tasks with or without stress. With respect to tasks natively running on the device, containerization impacts on the WCET overruns and missed deadlines only when the running RT tasks are equal or more than the CPU cores. This is due to the fact that when the task overruns the deadline and all CPU cores are allocated to RT tasks, the system cannot remap the task to a different available core.

More remarkably, we found that the observed WCET does not depend on the number of containers created to group the RT tasks. This is highlighted by the increased value of observed WCET (both with or without stress) by around 9.5% when the tasks are grouped into one container w.r.t. the native configuration, while the value is comparable between 1 container and n containers with any number of tasks. We observed the same behaviour for the WCET overruns and missed deadlines. The task WCET was overrun 28 times with 8 tasks in one Docker container, and this led to 24 missed deadlines over 8 millions (i.e., 0.00035%). The values do not increase by increasing the number of containers.

In summary, we noticed a negligible overhead when containerizing applications in general. Only at maximum utilization (e.g., 8 tasks over 8-CPU cores with 95% utilization) does the overhead lead to missed deadlines. Since such provisioning is rarely ever done, we contend that containerization with the proposed off-the-shelf technology can support real-time applications. In our experimental analysis, we obtained no missed deadlines for the containerized configurations also with eight tasks by slightly relaxing the deadline w.r.t. to the period (e.g., increasing the period from 4.0 to 4.5ms, with WCET 4.3ms, still with 95% maximum utilization per task).

B. Orchestration with RT support

To evaluate our orchestration platform with RT support, we setup a cluster with three nodes consisting of an Nvidia Jetson Xavier with Linux v4.9 with the *RT preempt_rt* patch (node J), an Intel i7 9700k locked at 3.6GHz and 16GB DRAM with Linux 5.10 with the *RT preempt_rt* patch (node I), and an AMD Ryzen 7 4800u CPU, 16GB DRAM with Linux v5.11 without *RT patch* (node R).

Orchestration platform configurations. We used three different configurations for the orchestration platform.

- (1) *Native K3S-standard*: It implements the standard Kubernetes configuration for orchestration.
- (2) *Native K3S-best configuration*: It implements a static *best* configuration for orchestration with the native Kubernetes scheduler. It represents the optimal task-to-node orchestration solution and is manually configured.
- (3) *Proposed RT-K3S*: Our proposed orchestration platform with the extended RT specifications and plugins.

Workload and deployment setups. The workload consists of 73 tasks. 72 of them have been containerized into 72 corresponding RT-containers. One task has been containerized into one standard (i.e., non RT) container and implements stress activity through memory accesses. We tested two deployment setups. The first consists of 32 containers with criticality level A, 24 with criticality level B, and 16 with criticality level C. The second test deployment is reversed with 16 containers with criticality level A, 24 with criticality level B, and 32 with C. All containers have one instance of “cyclicdeadline”, with the following utilizations: 12% for A, 16% for B, and 20% for C. The WCET has been set to 2.9ms for all tasks, and the deadlines at 24.2ms, 18.1ms and 14.5ms for levels A, B, and C, respectively.

Key results. Table II summarizes the workload and deployment setups in the first three columns. The fourth column shows the number of RT tasks that have been mapped into the corresponding (RT) nodes by the orchestrator but that have been rejected by the Linux kernel admission test (see eqn. 1). The last column shows, for all criticality levels, the number of missed deadlines. The column reports the deadline missed by the running tasks (not rejected) and the total missed deadlines (those missed by the running + those missed because of the task rejection).

We found that the orchestration of the standard Kubernetes produces a task-to-node mapping that is completely inadequate in case of RT tasks. This is underlined by the amount of RT tasks that, after mapping, have been rejected by the Linux Kernel RT admission test and by the number of missed deadlines w.r.t. the best (manually configured) orchestration configuration.

The results of the manual best configuration are achieved with the support for real-time containers (proposed RT-K3S).

We also evaluated the efficiency of the proposed solution, w.r.t. the native Kubernetes, with a dynamic workload and cluster. We set up a first scenario (first two rows of Table III) in which 16 RT

# tasks	config.	average runtime (ms)	average runtime with stress (ms)	Observed WCET (ms)	Observed WCET with stress (ms)	WCET overruns (#)	WCET overruns with stress (#)	missed deadlines (#)	missed deadlines with stress (#)	total deadlines per config (#)
1	native	1.219	1.256	3.143	3.655	0	0	0	0	30k
	1 container	1.164	1.172	3.055	3.119	0	0	0	0	
2	native	1.234	1.251	3.118	3.755	0	0	0	0	60k
	1 container	1.154	1.169	3.092	3.052	0	0	0	0	
	2 containers	1.144	1.162	3.091	3.090	0	0	0	0	
4	native	1.225	1.262	3.100	3.845	0	0	0	0	120k
	1 container	1.155	1.160	3.522	4.014	0	1	0	1	
	4 containers	1.134	1.154	3.783	3.704	0	0	0	0	
8	native	1.187	1.204	3.565	4.120	0	9	0	5	8M
	1 container	1.155	1.146	4.337	4.511	28	23	24	21	
	8 containers	1.140	1.141	4.405	4.322	26	18	10	14	

TABLE I: Comparison of RT-tasks running natively, containerized in a single and multiple Docker containers.

De- ployment	Config.	Critical tasks/ Nodes	Tasks rejected	Deadlines missed (running tasks - running+rejected)
1 st	Native K3S- standard	A: J(32)	A: 8	A: 3.3% - 27.5%
		B: J(24)	B: 7	B: 9.1% - 35.6%
		C: J(16)	C: 6	C: 6.5% - 41.5%
		oth.: J(1)		
	Native K3S-best config.	A: I(32)	A: 0	A: 0.0% - 0.0%
		B: I(24)	B: 0	B: 0.0% - 0.0%
2 nd	Native K3S- standard	C: J(16)	C: 0	C: 0.0% - 0.0%
		oth.: R(1)		
	Proposed RT-K3S	A: I(32)	A: 0	A: 0.0% - 0.0%
		B: I(24)	B: 0	B: 0.0% - 0.0%
	Native K3S-best config.	C: J(32)	C: 0	C: 0.0% - 0.0%
		oth.: R(1)		
3 rd	Native K3S- standard	A: I(16)	A: 0	A: 0.0% - 0.0%
		B: I(24)	B: 0	B: 0.0% - 0.0%
		C: J(32)	C: 0	C: 0.0% - 0.0%
		oth.: R(1)		
	Proposed RT-K3S	A: I(16)	A: 0	A: 0.0% - 0.0%
		B: I(24)	B: 0	B: 0.0% - 0.0%

TABLE II: Comparison among Kubernetes standard orchestration, best orchestration, and the proposed RT-K3S.

containers are deployed and run correctly onto the cluster for 100% RT utilization, while the total CPU utilization is 40% (instant T_0 in the table). In this context, there is CPU time available for the (standard) orchestrator, while the CPU is fully loaded for the RT admission test (see eqn. 1).

When a new RT container arrives during execution (instant $T_0 + t_\alpha$ in the table), the orchestration approach of the native Kubernetes deploys the container on the cluster. This leads to an overload of the cluster RT utilization (i.e., more than 100%) and, as a consequence, to a failed admission test. This failed container keeps getting restarted by Kubernetes. The admission test succeeds and the container is deployed only when RT resources of the same node become available and the new RT utilization is within 100%. Nevertheless, if any other different RT node becomes available, the issue persists as Kubernetes has no way of knowing the cause of the failure. In contrast, with the proposed solution, the new container remains pending *at orchestration level* until any RT resources (of the same or other nodes) become available.

Configuration	T_0		$T_0 + t_\alpha$	
	Deployed	Pending - Rejected	Deployed	Pending - Rejected
Native- K3S	$N_1: 8$	1 - 0	$N_1: 9$	0 - 1
	$N_2: 8$		$N_2: 8$	
Proposed RT-K3S	$N_1: 8$	1 - 0	$N_1: 8$	1 - 0
	$N_2: 8$		$N_2: 8$	
Native- K3S	$N_1: 8$	1 - 0	$N_1: 9$	0 - 1
	$N_2: 0$		$N_2: 0$	
Proposed RT-K3S	$N_1: 8$	1 - 0	$N_1: 8$	0 - 0
	$N_2: 0$		$N_2: 1$	

TABLE III: Experimental results for dynamic orchestration.

We set up a second scenario (last two rows of Table III), in which the first RT node has utilization 100%, with 8 containers, while the second RT node has utilization 0%. When a new RT container has to be deployed, in the standard Kubernetes, there is only a 50% chance for the correct node to be picked, as the orchestrator has no knowledge of the RT utilization. In contrast, the proposed orchestration platform guarantees that the correct node (one with 0% RT utilization) is always selected.

C. Autonomous mobile robot case study

Our case study consists of a Robotnik RB-Kairos, which is a skid-steering mobile platform equipped with an Universal Robots UR5 and a Schunk WSG50 gripper. The computing architecture is composed of two onboard programmable devices (i.e., an NVIDIA Jetson Xavier with the RT OS and a board with an i7 9700 locked at 3.0 GHz with 8GB DRAM and a *non real-time OS*). The edge-server consists of a desktop computer with an octa-core CPU and 16GB of ram, also with an RT kernel. The robot's mission is implemented through an 80-tasks (ROS nodes) SW application. It performs pick and place operations, delivering goods from a conveyor belt to a storage area and vice-versa. The most critical part of the software is the driver controlling the arm operations. This task needs to maintain a control loop that communicates directly with the arm hardware at 120Hz (8ms deadline, 7.6ms WCET, 90% utilization). Exceeding the deadline and delaying such a communication beyond a certain threshold causes the arm to go into safe mode and halting operations. This translates into a measured maximum lateness bound of 0.7ms, after which the connection is dropped.

As expected, we found that, with the native K3S orchestration, all tasks were mapped onto the three devices randomly. This led the missed deadlines to exceed the threshold and to the safety stop of the robot. We then evaluated two alternative scenarios with

Config.	Avg runtime (ms)	Std. dev.	Observed WCET (ms)	Max. lateness (ms)	Missed deadlines	
					arm driver	SLAM
Native	7.15	3.58	32.32	-	24.50%	-
non-RT + RT [27]	0.20	0.11	5.72	0.95	1.28%	≈ 0%
non-RT + A + B + C	0.15	0.07	2.06	0.49	0.17%	≈ 0%

TABLE IV: Experimental results with the RB-Kairos.

the proposed RT orchestrator. The first supports multiple criticality levels (i.e., non real time tasks + A, B, and C real-time tasks), while the second only supports non real-time + real-time tasks (e.g., [27]). In the first solution, we classified 3 tasks implementing the robot localization and mapping (SLAM) as RT criticality B. Then, the arm driver as an RT task with criticality C. We defined the edge server as an RT node for criticalities A and B, while the on-board Jetson for criticality C. In the second scenario, we classified the SLAM, as well as the arm driver, as RT. Both are deployed on the RT-capable Jetson onboard. In both scenarios the rest of the software was configured as non real-time. These two test cases allow us to evaluate the improvement for the automatic separation of different classes of criticality as well as the benefits of RT orchestration.

Table IV shows the results. We observed very different average runtime and WCET when comparing the native Kubernetes approach and the proposed RT-K3S, making the native solution practically unusable. Our solution based on multi-level criticality (third row in the table) manages to take advantage of criticality-aware load distribution, which allows the arm driver to perform substantially better than native and other alternatives, with lower observed WCET and reduced deadline misses. The much less restrictive control loop of the SLAM nodes allows for great performance in both solutions. Also, differently from native and other alternatives that incurred in high lateness of critical tasks and a consequent stop of the mobile robot, the proposed solution did not suffer from any safety-related stop as the maximum lateness bound was never exceeded. This is thanks to the isolation of the highest criticality tasks from the lower criticality ones implemented by the orchestrator.

V. CONCLUSION

This paper addressed the problem of containerization and orchestration of software for autonomous mobile robots, which is characterized by mixed criticality tasks deployed across Edge-Cloud architecture. It presented an orchestration platform (RT-K3S) for mixed-criticality systems that extends the standard Kubernetes to support RT containers. It presented a characterization of the container overhead when real-time tasks are deployed on off-the-shelf embedded systems (e.g., Linux with *preempt-rt* patch on NVIDIA Jetson). The paper evaluated the efficiency of the proposed solution with both synthetic benchmarks and with a software implementing the mission of a Robotnik RB-Kairos mobile robot.

REFERENCES

- [1] D. Merkel *et al.*, “Docker: lightweight linux containers for consistent development and deployment,” *Linux journal*, vol. 2014, no. 239-art.2, 2014.
- [2] P. Barham, B. Dragovic, K. Fraser, S. Hand, T. Harris, A. Ho, R. Neugebauer, I. Pratt, and A. Warfield, “Xen and the art of virtualization,” in *Operating Systems Review (ACM)*, vol. 37, no. 5, 2003, pp. 164–177.
- [3] M. Chae, H. Lee, and K. Lee, “A performance comparison of linux containers and virtual machines using docker and kvm,” *Cluster Computing*, vol. 22, pp. 1765–1775, 2019.
- [4] M. Shaik and *et al.*, “Enabling fog-based industrial robotics systems,” in *IEEE Symposium on Emerging Technologies and Factory Automation, ETFA*, vol. 2020-September, 2020, pp. 61–68.
- [5] AUTOSAR: AUTomotive Open System ARchitecture, “The standardized software framework for intelligent mobility,” 2021, www.autosar.org.
- [6] Collins Aerospace, “Connectivity and network services,” 2021, www.arinc.com.
- [7] M. Sollfrank, F. Loch, S. Denteneer, and B. Vogel-Heuser, “Evaluating docker for lightweight virtualization of distributed and time-sensitive applications in industrial automation,” *IEEE Transactions on Industrial Informatics*, vol. 17, no. 5, pp. 3566–3576, 2021.
- [8] A. Burns and R. Davis, “Mixed criticality systems-a review,” *Dept. of Computer Science, University of York, Tech. Rep.*, pp. 1–69, 2013.
- [9] —, “A survey of research into mixed criticality systems,” *ACM Computing Surveys*, vol. 50, no. 6, 2017.
- [10] T. Tasci, J. Melcher, and A. Verl, “A container-based architecture for real-time control applications,” in *2018 IEEE International Conference on Engineering, Technology and Innovation, ICE/ITMC*, 2018.
- [11] M. Cinque, R. Corte, A. Eliso, and A. Pecchia, “Rt-cases: Container-based virtualization for temporally separated mixed-criticality task sets,” in *Leibniz International Proc. in Informatics*, vol. 133, 2019.
- [12] L. Abeni, A. Balsini, and T. Cucinotta, “Container-based real-time scheduling in the linux kernel,” *SIGBED Rev.*, vol. 16, no. 3, p. 33–38, Nov. 2019.
- [13] M. Thiyyakat, S. Kalambur, and D. Sitaram, “Improving resource isolation of critical tasks in a workload,” *Lecture Notes in Computer Science*, vol. 12326 LNCS, pp. 45–67, 2020.
- [14] D. Bernstein, “Containers and cloud: From lxc to docker to kubernetes,” *IEEE Cloud Computing*, vol. 1, no. 3, pp. 81–84, 2014.
- [15] C. Dupont, R. Giaffreda, and L. Capra, “Edge computing in iot context: Horizontal and vertical linux container migration,” in *In Proc. of Global Internet of Things Summit*, 2017.
- [16] Y. Xiong, Y. Sun, L. Xing, and Y. Huang, “Extend cloud to edge with kubernetes,” in *In Proc. of ACM/IEEE Symposium on Edge Computing*, 2018, pp. 373–377.
- [17] The Chief I/O, “How kubernetes is shaping the future of cars,” 2021, <https://thechief.io/c/editorial/how-kubernetes-is-shaping-the-future-of-cars/>.
- [18] F. Carpio, M. Delgado, and A. Jukan, “Engineering and experimentally benchmarking a container-based edge computing system,” *IEEE International Conference on Communications*, Jun 2020.
- [19] S. Aldegheri, N. Bombieri, S. Germiniani, F. Moschin, and G. Pravadelli, “A containerized ros-compliant verification environment for robotic systems,” in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2021, pp. 222–225.
- [20] F. Lumpp, M. Panato, F. Fummi, and N. Bombieri, “A container-based design methodology for robotic applications on kubernetes edge-cloud architectures,” in *Forum on specification Design Languages*, 2021.
- [21] A. Moga, T. Sivanthi, and C. Franke, “Os-level virtualization for industrial automation systems: Are we there yet?” in *Proc. of the ACM Symposium on Applied Computing*, 2016, pp. 1838–1843.
- [22] P. González-Nalda, I. Etxeberria-Agiriano, I. Calvo, and M. Otero, “A modular cps architecture design based on ros and docker,” *International Journal on Interactive Design and Manufacturing*, vol. 11, no. 4, pp. 949–955, 2017.
- [23] V. Struhár, M. Behnam, M. Ashjaei, and A. V. Papadopoulos, “Real-Time Containers: A Survey,” in *Workshop on Fog Computing and the IoT*, 2020, pp. 7:1–7:9.
- [24] F. Hofer, M. A. Sehr, A. Iannopollo, I. Ugalde, A. Sangiovanni-Vincentelli, and B. Russo, “Industrial control via application containers: Migrating from bare-metal to iaaS,” in *In Proc. of IEEE CloudCom*, 2019, pp. 62–69.
- [25] F. Hofer, M. A. Sehr, A. Sangiovanni-Vincentelli, and B. Russo, “Odre workshop: Probabilistic dynamic hard real-time scheduling in hpc,” 2020.
- [26] L. De Simone and G. Mazzeo, “Isolating real-time safety-critical embedded systems via sgx-based lightweight virtualization,” in *Proc. of International Symposium on Software Reliability Engineering Workshops, ISSREW 2019*, 2019, pp. 308–313.
- [27] V. Struhár, S. S. Craciunas, M. Ashjaei, M. Behnam, and A. V. Papadopoulos, “React: Enabling real-time container orchestration,” in *2021 26th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2021, pp. 1–8.